



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • START THE CLUSTER • STEPS 1 AND 2

1 ☐ Create a container that is a Kubernetes cluster

```
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
-p 6443:6443 rancher/k3s:v1.31.5-k3s1 server --disable=traefik
```

<code>-d</code>	Detached. It runs in the background and you get your prompt back.
<code>--name k8s</code>	A name of your choosing. Without it, Docker still identifies the container, but by its id: 8d7726f96f747554440565a92a034cda1f767726619081595bab18cfd76987 Every later command would have to carry that, or its short form 8d7726f96f74 . k8s is easier to type and easier to get right.
<code>--privileged</code>	Lifts the usual restrictions. k3s mounts cgroups and runs its own container runtime inside this container.
<code>--tmpfs /run</code>	Two small writable in-memory filesystems that k3s needs (also /var/run).
<code>-p 6443:6443</code>	HOST:CONTAINER. Port 6443 on your laptop is forwarded to port 6443 inside the container. The left one is yours, the right one is the container's. 6443 is where the Kubernetes API server listens.
<code>rancher/k3s:v1.31.5-k3s1</code>	The image. rancher/k3s is the repository, v1.31.5-k3s1 is the tag. You named no registry, so Docker went to Docker Hub and fetched it from there.
<code>server</code>	Not a docker flag. Everything after the image name is the command handed to the program inside. server tells k3s to be the control plane.
<code>--disable=traefik</code>	Also k3s's, not docker's. Traefik is an ingress controller : it routes HTTP arriving from outside to services inside. It claims ports 80 and 443 and adds startup time, and nothing today arrives from outside.

2 ☐ Step inside the container you just made

```
docker exec -it k8s sh
```

`-i` keeps the input open, `-t` gives you a terminal. Together they get you an **interactive shell running inside that container** — not on your laptop. Your prompt changes; that is the sign.

Prove it before you go on. Type `hostname` and write down what it says: _____

Keep that value. It comes back in panel 3, and it is not a coincidence. `exit` leaves the shell; the container keeps running without you.

Do not close this shell.

Steps 3 to 6 and every `kubectl` in panels 5 to 9 are typed **here**, at this prompt. If you type them on your laptop instead, you will get `kubectl: command not found` — the cluster's tools live in the container, not on your machine.

Closed it by accident? `docker exec -it k8s sh` puts you straight back. Nothing is lost: the cluster never stopped.

2 · DRIVE THE CLUSTER · STEPS 3 TO 6

Type these at the shell from step 2, inside the container.

3 ☐ Ask the cluster to keep a whale running

```
kubectl create deployment whale --image=ghcr.io/oesteban/whalesay --replicas=3
```

A **Deployment** says: this many of these should exist, at all times. You do not say when to start anything.

Why is the registry spelled out here, when step 1 named none?

Because there are two copies of this image and they are **not the same thing**. Run these two *on your laptop*, not in the cluster shell:

```
docker buildx imagetools inspect ghcr.io/oesteban/whalesay
docker buildx imagetools inspect oesteban/whalesay
```

The first prints a Manifests: list with Platform: linux/amd64 **and** Platform: linux/arm64 . The second prints no list at all: it is a single image, linux/amd64 only.

An Apple Silicon Mac runs an **arm64** Linux VM, and k3s's containerd has no emulation to fall back on. The Docker Hub copy would give `exec format error` and the pod would never start. The GHCR copy was built for both by GitHub Actions on Friday.

On Linux or Windows, on Intel or AMD: the Docker Hub copy runs perfectly well. Check the manifests above, then try it alongside: `kubectl create deployment hub --image=oesteban/whalesay --replicas=1` Which architecture is your laptop?

4 ☐ Look at what the controller made

```
kubectl get pods — once straight away, once about a minute later.
```

This lists the **Pods** the Deployment created on your behalf. Two columns matter: **STATUS**, what the pod is doing right now, and **RESTARTS**, how many times the container inside it has been started again.

5 ☐ Ask for the same image to run once, instead of forever

```
kubectl create job whale --image=ghcr.io/oesteban/whalesay -- cowsay 'a cluster ran me'
```

A **Job** says: run this to completion, once. Same image, same cluster, same command — only the kind of object is different.

6 ☐ Read what it printed

```
kubectl logs job/whale
```

```
kubectl logs
```

 prints whatever a container wrote to its output. You did not have to find the pod: naming the Job was enough.

3 · WHAT YOU STARTED

From `kubectl get nodes`:

NAME	VERSION
STATUS	AGE when first Ready

4 · THE TWO READINGS, AND THE REASON

First reading

STATUS	
RESTARTS	

A minute later

STATUS	
RESTARTS	

Compare the NAME in panel 3 with the `hostname` you wrote in step 2.

The container did not crash. It printed its whale and exited **successfully**. So why is Kubernetes starting it again?

The Job in step 5 used **the same image** and did not do this. What did you change, in one sentence?

5 • WHAT THE LOGS WILL TELL YOU

Logs are the first place you look when something is wrong, and they are not only for things that are wrong. Try all four, inside the cluster.

<code>kubectl logs deploy/whale</code>	There are three pods. Which one did it pick, and how do you know?
<code>kubectl logs <pod> --previous</code>	The container that already died . Why would that ever be the only copy left?
<code>kubectl logs -l app=whale --prefix</code>	All three at once. What does the prefix tell you?
<code>kubectl logs <pod> --timestamps</code>	Now you can see when . How far apart are the restarts?

A pod is gone and you want to know why. Which of those four do you reach for, and what would you have lost if you had waited an hour?

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus.
Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • NOW USE THE IMAGE YOU BUILT ON FRIDAY

Everything so far ran **someone else's** image. Repeat step 2 with your own, the one your GitHub Actions workflow published on Friday:

```
kubectl create deployment mine --image=ghcr.io/<your-handle>/whalesay
```

Your handle _____ Did it pull? _____

If it did not, `kubectl describe pod` will name the reason. Write it down, then fix it.

Nobody told the cluster where `ghcr.io` is, and nobody gave it a password. So: what **is** a registry, and what had to be true of your package for this to work at all?

On Friday you pushed to **GHCR**. When you type `docker pull alpine` you get an image from **Docker Hub**, and you never type a registry name. Where did that default come from, and what is the full name your `alpine` actually has?

7 • MAKE THE CLUSTER EXPLAIN ITSELF

The node's NAME is not a hostname anybody chose. Where does it come from, and what does that tell you about what a "node" is here?

`kubectl describe pod <one of the whale pods>` and read the **Events** block at the very bottom. Copy the two most recent event lines.

Then `kubectl get events --sort-by=.lastTimestamp | tail -5`. Whose account of the morning is this — yours, or the controller's?

8 • TWO IMAGE STORES, ONE NAME

Inside the cluster: `crictl images` .

On your laptop, in another terminal: `docker images` .

whalesay in `crictl` ? _____ in `docker` ? _____

Nobody copied anything from your laptop into the cluster. So how did the image get there?

9 • A WISH IS NOT A COMMAND

`kubectl scale deployment whale --replicas=5`, wait, then `kubectl scale deployment whale --replicas=0`. Count the pods after each.

pods at 5 _____ pods at 0 _____

Now delete the **Job's** pod: `kubectl delete pod <the job pod>`. Does it come back? Does a Deployment's pod come back? Why are the answers different?

10 • EXTRA MILE • SATISFY THE DEPLOYMENT INSTEAD OF ABANDONING IT

In step 5 you stopped the restarting by changing the **object**. There is a second way that leaves it a Deployment: same image, no rebuild, no re-push, sitting at `RESTARTS 0` and `Running`. Write the command.

You have now satisfied a Deployment two ways. Which would you use for a web server, and which for a nightly report?

No hint. The image runs one program and then stops; nothing says it has to.

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.